

Docker und Nix im Vergleich: Metriken und Bewertung von Deployment-Verfahren

Projektbericht im Studiengang WDS125

Verfasser: Leon Feuerstein · Modul: Full Stack Handwerk · Betreuung: Prof. Dr.-Ing. Mark Schutera

Kurzfassung

Die Vorlesung bringt die Beispielanwendung PixelWise von Hand auf eine Produktions-VM, wodurch die Umgebung nicht reproduzierbar ist. Diese Arbeit ersetzt das manuelle Deployment durch zwei Verfahren, Containerisierung mit Docker und deklarative Builds mit Nix, und vergleicht sie an identischem Quellcode. Drei Konfigurationen werden gebaut und vermessen: ein naiver Docker-Build, ein optimierter Docker-Build und ein Nix-Build mit OCI-Ausgabe. Als Messgrößen dienen Build-Zeit, Image-Größe, bit-genaue Reproduzierbarkeit und Konfigurationsaufwand. Der optimierte Docker-Build liefert das kleinste Image (321 MB) und baut inkrementell am schnellsten (4,1 s), während allein der Nix-Build bit-identisch reproduzierbar ist. Da beide Werkzeuge unterschiedliche Schichten bedienen, lassen sich ihre Stärken kombinieren: Nix erzeugt das OCI-Image deterministisch, Docker betreibt es.

1 Ausgangslage und Zielsetzung

Der Kursblock *Development and Production* trennt eine Entwicklungs- und eine Produktions-VM und bringt PixelWise, eine Ziffernerkennung aus Frontend, Backend und Datenbank, am Ende manuell über SSH auf die Produktions-VM. Die Abhängigkeiten werden direkt im Betriebssystem installiert. Eine später neu aufgesetzte VM erhält je nach Paketquelle und Zeitpunkt abweichende Versionen, sodass sich ein ausgelieferter Stand nicht zuverlässig rekonstruieren lässt. Die Vorlesung führt dies als „runs on my machine“ und verweist auf die dev/prod-Parität als Ziel [8][9].

Containerisierung und deklarative Paketverwaltung adressieren dieses Problem auf verschiedene Weise. Docker bündelt Anwendung und Laufzeitumgebung in einem Image, Nix beschreibt den Build als Funktion fixierter Eingaben. Die vorliegende Arbeit setzt beide Verfahren an dieselbe Referenzanwendung, misst sie über definierte Metriken und leitet daraus ab, für welche Anforderung welches Verfahren trägt. Der praktische Teil, also Anwendung, Konfigurationen und Messskripte, ist als lauffähiges Repository hinterlegt.

2 Technische Grundlagen

2.1 Docker und das OCI-Format

Docker isoliert Prozesse über Linux-Namespaces und cgroups und legt eine Anwendung als Stapel schreibgeschützter Layer ab, die zur Laufzeit von einem Overlay-Dateisystem überlagert werden. Gebaut wird über ein Dockerfile, dessen Anweisungen einzeln im Layer-Cache landen. Das Imageformat ist nicht an Docker gebunden: Die Open Container Initiative standardisiert seit 2015 Aufbau und Laufzeitverhalten, sodass jede konforme Laufzeit dasselbe Image ausführen kann [3]. Reproduzierbarkeit ist hier kein Standardverhalten. Befehle wie `apt` oder `pip` ziehen tagesaktuelle Versionen, und eingebettete Zeitstempel ändern sich pro Lauf; byte-gleiche Builds verlangen laut Dokumentation BuildKit ab Version 0.11 mit `SOURCE_DATE_EPOCH` und über Digests gepinnte Basis-Images [4][5].

2.2 Nix

Nix behandelt jeden Build als Funktion seiner Eingaben und benennt jede Ableitung über einen Hash aus Quelltext, Abhängigkeiten und Build-Anweisungen; eine `flake.lock` fixiert die Eingaben exakt [1][7]. Der Build läuft in einer Sandbox, weshalb identische Eingaben identische Artefakte erzeugen; `nix build -rebuild` prüft das. Eine Analyse von rund 710.000 nixpkgs-Paketen weist bitweise Reproduzierbarkeitsraten von 69 bis 91 Prozent nach, mit steigender Tendenz [2]. Über `dockerTools.buildLayeredImage` gibt Nix das Ergebnis als OCI-Image aus, das in derselben Laufzeit wie ein Docker-Image läuft. Damit überlappen die beiden Werkzeuge genau an der Schnittstelle, an

der dieser Vergleich ansetzt.

3 Versuchsaufbau

Damit sich nur das Bereitstellungsverfahren unterscheidet, liegt allen Konfigurationen dieselbe Anwendung zugrunde: ein REST-Backend (Python 3.12, FastAPI), ein Softmax-Klassifikator in reinem NumPy (92,6 Prozent Test-Genauigkeit auf MNIST), ein statisches Canvas-Frontend und PostgreSQL zur Persistenz. PostgreSQL ist überall dasselbe offizielle Image und damit nicht Teil des Vergleichs; gebaut und gemessen wird das Backend-Image. Quelltext und Abhängigkeitsversionen bleiben konstant.

Verglichen werden drei Konfigurationen:

- K1, naiver Docker-Build: volles Basis-Image (`python:3.12`), ein Stage, Quellcode vor der Installation kopiert.
- K2, optimierter Docker-Build: Multi-Stage, schlankes Basis-Image (`python:3.12-slim`), Abhängigkeiten vor dem Code für einen wirksamen Layer-Cache, `.dockerignore`, nicht-root Laufzeit.
- K3, Nix-Build: Abhängigkeiten und Build deklarativ in `flake.nix`, Ausgabe als OCI-Image über `dockerTools.buildLayeredImage`.

Als Messgrößen dienen die Build-Zeit bei leerem Cache und nach einer minimalen Codeänderung, die Image-Größe komprimiert und on-disk, die byte-genaue Reproduzierbarkeit und der Konfigurationsaufwand in Code-Zeilen. Reproduzierbarkeit gilt als erfüllt, wenn zwei unabhängige Builds dasselbe Artefakt liefern, bei Docker über den Vergleich der Image-ID, bei Nix über `nix build -rebuild`. Als Kontrolle dient die funktionale Äquivalenz: Liefern alle Images auf gleicher Eingabe dieselbe Vorhersage, gehen Unterschiede allein auf das Verfahren zurück. Build-Zeiten werden mehrfach gemessen und als Mittelwert mit Standardabweichung berichtet. Messumgebung ist Docker 29.5.1 (containerd, overlayfs, klassischer Builder) und Nix 2.20.6 (nixpkgs 24.11) auf x86_64-linux. Absolute Größen hängen vom Storage-Treiber ab, weshalb die Verhältnisse zwischen den Konfigurationen maßgeblich sind.

4 Ergebnisse

Alle drei Images bauen erfolgreich und liefern auf gleicher Eingabe dieselbe Vorhersage (Ziffer 5 bei 0,4204 Konfidenz); die funktionale Äquivalenz ist damit bestätigt. Tabelle 1 enthält die Messwerte, Abbildung 1 stellt die beiden aussagekräftigsten Größen gegenüber.

Tabelle 1: Messergebnisse je Konfiguration. Build-Zeiten als Mittelwert $\pm$ Standardabweichung. Image-Größe komprimiert (docker image inspect) und unkomprimiert on-disk.

Messgröße	K1 (naiv)	K2 (optimiert)	K3 (Nix)
Build-Zeit kalt [s]	22,8 $\pm$ 0,1	27,4 $\pm$ 0,1	88,0
Build-Zeit inkrementell [s]	22,1 $\pm$ 0,3	4,1 $\pm$ 0,03	6,1
Image-Größe komprimiert [MB]	455	76,6	390
Image-Größe on-disk [MB]	1760	321	796
Reproduzierbar (bit-identisch)	nein	nein	ja
Konfigurationsaufwand [LOC]	7	18	45

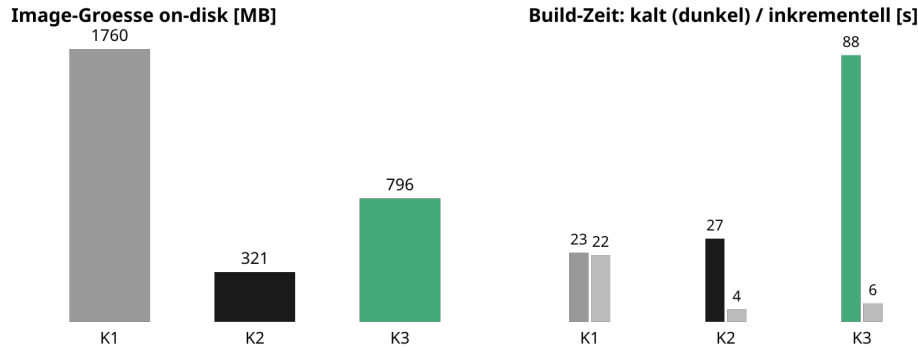


Abbildung 1: Image-Größe on-disk (links) sowie Build-Zeit kalt und inkrementell (rechts). K2 ist am kleinsten und iteriert am schnellsten, K3 hat den teuersten Erstaufbau.

Bei der Reproduzierbarkeit erzeugen zwei aufeinanderfolgende Builds von K1 und K2 unterschiedliche Image-IDs; beide Docker-Varianten sind ohne Zusatztooling nicht bit-identisch, was sich mit der Dokumentation deckt [4]. Der Nix-Build ist ohne weitere Konfiguration reproduzierbar: `nix build -rebuild` baut das Image erneut und bestätigt Bit-Gleichheit. Das im Messsystem fehlende BuildKit verhindert den reproduzierbaren Docker-Build, belegt aber zugleich, dass Docker diese Eigenschaft nicht von sich aus mitbringt.

Bei Größe und Geschwindigkeit führt der optimierte Docker-Build. Das Multi-Stage-Verfahren hält die Build-Werkzeuge aus dem finalen Image, wodurch K2 mit 321 MB on-disk um den Faktor 5,5 kleiner ausfällt als K1 (1760 MB). Die Cache-freundliche Layer-Reihenfolge baut nach einer Codeänderung nur den geänderten Teil neu, sodass K2 inkrementell 4,1 s statt 22,1 s bei K1 benötigt. K3 liegt mit 796 MB zwischen beiden, da das Nix-Image die vollständige Abhängigkeits-Closure samt Interpreter und glibc enthält; sein Erstaufbau dauert mit 88 s am längsten, weil er diese Closure lädt, fällt inkrementell aus dem Store aber auf 6,1 s.

5 Bewertung

Die Messung trennt die Verfahren nach Einsatzzweck. K2 dominiert bei Image-Größe und Iterationsgeschwindigkeit, weil Multi-Stage die Build-Werkzeuge aus dem Image entfernt und die Cache-freundliche Layer-Reihenfolge nur den geänderten Code neu baut. K3 ist die einzige Variante mit reproduzierbarem Build, erkaufte dies aber mit einem gegenüber K2 deutlich größeren Image (796 statt 321 MB) und dem höchsten Konfigurationsaufwand (45 statt 7 bis 18 Code-Zeilen, wobei LOC nur ein grober Komplexitätsindikator ist [6]). K1 bietet außer dem geringsten Schreibaufwand keinen Vorteil und taugt nur als kurzlebiger Prototyp. Zu beachten ist, dass Nix wie Docker eine zusätzliche Abhängigkeit ist, die installiert sein muss; Docker ist in Entwicklungs- und CI-Umgebungen jedoch häufiger bereits vorhanden, was den Einstieg in K1 und K2 senkt.

Reproduzierbarkeit ist dabei kein Selbstzweck, sondern hat einen konkreten Nutzen. Ein bit-identisches Image lässt sich gegen den Quelltext verifizieren, was Manipulationen in der Lieferkette aufdeckt; ein früherer Stand wird exakt rekonstruierbar, etwa für Audits oder einen Rückbau; und Build-Caches greifen verlässlich, weil gleiche Eingaben gleiche Ergebnisse liefern. Der Preis ist, dass jede Eingabe gepinnt und jede Zeitabhängigkeit kontrolliert werden muss. Wo Auditierbarkeit nicht gefordert ist, genügt funktionale Reproduzierbarkeit, und der schlankere Docker-Build ist die wirtschaftlichere Wahl. Tabelle 2 ordnet die Konfigurationen den typischen Anforderungen zu.

Tabelle 2: Empfehlung je Anforderung, begründet aus den Messwerten in Tabelle 1.

Anforderung	Empfehlung	Begründung
Reproduzierbare, auditierbare Auslieferung	Nix (K3)	einzige bit-identische Variante
Kleinstes Produktions-Image	Docker optimiert (K2)	321 MB, Faktor 5,5 unter K1
Häufige Iteration im Team	Docker optimiert (K2)	inkrementell 4,1 s statt 22,1 s
Geringe Werkzeugkette, schneller Start	Docker (K1 oder K2)	7 bis 18 LOC, meist vorinstalliert
Determinismus mit Standard-Betrieb	Nix-Build, Docker-Betrieb	OCI-Image aus Nix in üblicher Laufzeit

Da Docker und Nix unterschiedliche Schichten bedienen, schließen sie sich nicht aus. Nix erzeugt das OCI-Image deterministisch, Docker oder Kubernetes betreiben es in der Standard-Laufzeit. Diese Kombination verbindet die Reproduzierbarkeit des Nix-Builds mit der Verbreitung des Docker-Ökosystems und ist für Produktionsumgebungen mit Audit-Anforderungen die belastbarste Variante.

6 Herausforderungen, Stand und Ausblick

Bei der Umsetzung traten zwei technische Hürden auf. Die Image-Größe wird je nach Storage-Treiber unterschiedlich verbucht, einmal als komprimierter Inhalt, einmal entpackt on-disk; beide Werte werden deshalb getrennt erhoben und für die Bewertung auf Verhältnisse bezogen. Zudem greift eine Nix-Flake nur auf Dateien im eigenen Verzeichnisbaum zu, weshalb eine eigene Ableitung die Build-Closure auf die Laufzeitdateien beschränkt und die Trainingsdaten aus dem Image ausschließt.

Funktionsfähig ist das System vollständig: Alle drei Images bauen und laufen, liefern identische Vorhersagen und starten per `docker compose` samt Datenbank. Die Aussagekraft bleibt dennoch begrenzt, da nur eine Anwendung auf einem Host und ohne BuildKit gemessen wurde. Offen für eine Vertiefung sind ein bit-gleicher Docker-Build mit BuildKit und `SOURCE_DATE_EPOCH` im direkten Vergleich, ein CVE-Scan je Image (etwa mit Trivy) als belastbare Sicherheitsbewertung sowie eine CI-Integration mit Cache-Strategien.

7 Fazit

Für minimale Image-Größe und schnelle Iteration ist der optimierte Docker-Build überlegen, für reproduzierbare und auditierbare Auslieferung der Nix-Build. Als Produktionslösung empfiehlt sich die Kombination: ein deterministischer Nix-Build, betrieben über das Docker- und OCI-Ökosystem.

Literaturverzeichnis

- [1] Dolstra, E. (2006): *The Purely Functional Software Deployment Model*. Dissertation, Universiteit Utrecht.
- [2] *Does Functional Package Management Enable Reproducible Builds at Scale?* (2025). arXiv:2501.15919.
- [3] Open Container Initiative: *Image Format Specification*. specs.opencontainers.org/image-spec/.
- [4] moby/BuildKit: *Build reproducibility (SOURCE_DATE_EPOCH)*. [github.com/moby/buildkit, docs/build-repro.md](https://github.com/moby/buildkit/blob/main/docs/build-repro.md).
- [5] Reproducible Builds Project: *SOURCE_DATE_EPOCH*. reproducible-builds.org/docs/source-date-epoch/.
- [6] McCabe, T. J. (1976): *A Complexity Measure*. IEEE Transactions on Software Engineering, SE-2(4), S. 308 bis 320.
- [7] NixOS Foundation: *Nix Reference Manual: Store Paths*. nix.dev/manual/nix.
- [8] Wiggins, A. (2017): *The Twelve-Factor App, Faktor X: Dev/prod parity*. 12factor.net.
- [9] Schutera, M. (2026): *Full Stack Handwerk: Development and Production*. Vorlesungsskript.